Ejercicio 1

Quiero implementar en Java una pila de numeros enteros utilizando una lista enlazada simple sin usar estructuras de la libreria estandar de Java. La estructura debe modelarse con una clase Nodo interna o separada y una clase PilaEnlazada. El problema exige comportamiento LIFO. El puntero head representa directamente el tope de la pila ylas operaciones principales push(int x) y pop() se resuelven en tiempo constante O(1), ya que ambas agregan y quitan nodos exclusivamente en la cabeza reconectando una sola referencia sin recorrer nada. La pila debe implementar los metodos: push, pop, peek, isEmpty, contains(int valor) e imprimir. Casos especiales a contemplar: lanzar excepcion de Underflow si intentan hacer pop o peek con la pila vacia (head == null). Para las operaciones push, pop, peek e isEmpty se espera una complejidad de O(1), mientras que contains e imprimir deben ser O(n) al tener que recorrer nodo a nodo. Incluir comentarios y probar apilando 3 numeros, viendo el tope, buscando un valor existente/inexistente y desapilando todo.

```
import java.util.Arrays;


public class ColaLineal {


    private int[] arreglo;

    private int front;

    private int rear;


    public ColaLineal(int capacidad) {

        arreglo = new int[capacidad];

        front = -1;

        rear = -1;

    }


    // Inserta un elemento al final de la cola

    public void enqueue(int elemento) {
```

```
    // Problema: solo comprobamos si rear llegó al final
    if (rear == arreglo.length - 1) {
        System.out.println(
            "ERROR: La cola esta llena. No se puede insertar "
            + elemento + "."
        );
        return;
    }

    // Si la cola estaba vacía, inicializamos front
    if (front == -1) {
        front = 0;
    }

    rear++;
    arreglo[rear] = elemento;

    System.out.println("Enqueue(" + elemento + ") realizado.");
}

// Retira el elemento del frente
public int dequeue() {

    if (front == -1) {
        System.out.println("ERROR: La cola esta vacia.");
```

```
        return -1;
    }

    int elemento = arreglo[front];

    // Si era el último elemento, la cola queda vacía
    if (front == rear) {
        front = -1;
        rear = -1;
    } else {
        front++;
    }

    return elemento;
}

// Muestra el estado interno de la cola
public void mostrarEstado() {
    System.out.println("Arreglo: " + Arrays.toString(arreglo));
    System.out.println("front = " + front);
    System.out.println("rear = " + rear);

    int cantidad = (front == -1) ? 0 : rear - front + 1;
    System.out.println("Cantidad real de elementos = " + cantidad);
    System.out.println();
}
```

```
public static void main(String[] args) {

    ColaLineal cola = new ColaLineal(3);

    System.out.println("=== PASO 1: LLENADO INICIAL ===");

    cola.enqueue(1);
    cola.enqueue(2);
    cola.enqueue(3);

    cola.mostrarEstado();

    System.out.println("=== PASO 2: LIBERAR EL FRENTE ===");

    System.out.println("Dequeue: " + cola.dequeue());
    System.out.println("Dequeue: " + cola.dequeue());

    cola.mostrarEstado();

    System.out.println("=== PASO 3: INTENTAR INSERTAR 4 ===");

    cola.enqueue(4);

    cola.mostrarEstado();
}
```

}

=== PASO 1: LLENADO INICIAL ===

Enqueue(1) realizado.

Enqueue(2) realizado.

Enqueue(3) realizado.

Arreglo: [1, 2, 3]

front = 0

rear = 2

Cantidad real de elementos = 3

=== PASO 2: LIBERAR EL FRENTE ===

Dequeue: 1

Dequeue: 2

Arreglo: [1, 2, 3]

front = 2

rear = 2

Cantidad real de elementos = 1

=== PASO 3: INTENTAR INSERTAR 4 ===

ERROR: La cola esta llena. No se puede insertar 4.

Arreglo: [1, 2, 3]

front = 2

rear = 2

Cantidad real de elementos = 1

Ejercicio 2

Necesito implementar en Java una cola de numeros enteros utilizando una lista enlazada simple. La estructura corresponde modelarla como cola porque debe respetar el principio FIFO. La cola necesita dos referencias, head (frente de extraccion) y tail (final de insercion). Si solamente usaramos head aparece un problema de rendimiento: para encolar habria que recorrer toda la lista hasta el ultimo nodo degradando la insercion a O(n), mientras que con tail se conecta directo en tiempo constante O(1). Debe implementar las operaciones: enqueue(), dequeue(), front(), isEmpty(), contains() e imprimir. Casos especiales a contemplar: al desencolar el ultimo elemento existente (head pasa a ser null), tail tambien debe actualizarse obligatoriamente a null para evitar referencias a nodos fantasmas; y controlar el vacio al intentar hacer dequeue o front. Probar encolando elementos, consultando el frente y vaciando la cola.

```
public class ColaEnlazada {


    // Nodo de la lista enlazada
    private static class Nodo {
        int dato;
        Nodo sig;


        public Nodo(int dato) {
            this.dato = dato;
            this.sig = null;
        }
    }


    // head: frente de extraccion
    // tail: final de insercion
    private Nodo head;
    private Nodo tail;
```

```
public ColaEnlazada() {
    head = null;
    tail = null;
}

// Comprueba si la cola esta vacia: O(1)
public boolean isEmpty() {
    return head == null;
}

// Agrega un elemento al final de la cola: O(1)
public void enqueue(int x) {
    Nodo nuevo = new Nodo(x);

    if (isEmpty()) {
        // Si esta vacia, el nuevo nodo es frente y final
        head = nuevo;
        tail = nuevo;
    } else {
        // Conectamos el antiguo ultimo con el nuevo nodo
        tail.sig = nuevo;

        // Actualizamos el final
        tail = nuevo;
    }
}
```

```
// Quita y devuelve el elemento del frente: O(1)
public int dequeue() {
    if (isEmpty()) {
        throw new java.util.NoSuchElementException(
            "Underflow: no se puede desencolar una cola vacia."
        );
    }

    int valor = head.dato;

    // El segundo nodo pasa a ser el nuevo frente
    head = head.sig;

    // Si quitamos el ultimo nodo, la cola queda vacia
    if (head == null) {
        tail = null;
    }

    return valor;
}

// Consulta el frente sin quitarlo: O(1)
public int front() {
    if (isEmpty()) {
        throw new java.util.NoSuchElementException(
```

```
            "Underflow: no se puede consultar una cola vacia."
        );
    }

    return head.dato;
}

// Busca un valor recorriendo la cola: O(n)
public boolean contains(int valor) {
    Nodo actual = head;

    while (actual != null) {
        if (actual.dato == valor) {
            return true;
        }

        actual = actual.sig;
    }

    return false;
}

// Imprime desde el frente hasta el final: O(n)
public void imprimir() {
    if (isEmpty()) {
        System.out.println("Cola vacia.");
```

```
        return;
    }

    Nodo actual = head;

    System.out.print("Frente -> ");

    while (actual != null) {
        System.out.print(actual.dato);

        if (actual.sig != null) {
            System.out.print(" -> ");
        }

        actual = actual.sig;
    }

    System.out.println(" <- Final");
}

public static void main(String[] args) {

    ColaEnlazada cola = new ColaEnlazada();

    System.out.println("=== ENCOLAR ELEMENTOS ===");
```

```
cola.enqueue(10);
cola.enqueue(20);
cola.enqueue(30);

cola.imprimir();

System.out.println("\n=== CONSULTAR EL FRENTE ===");
System.out.println("Frente: " + cola.front());

System.out.println("\n=== BUSCAR ELEMENTOS ===");
System.out.println("¿Contiene 20? " + cola.contains(20));
System.out.println("¿Contiene 99? " + cola.contains(99));

System.out.println("\n=== DESENCOLAR ELEMENTOS ===");

while (!cola.isEmpty()) {
    System.out.println("Sale: " + cola.dequeue());
    cola.imprimir();
}

System.out.println("\n¿La cola esta vacia? " + cola.isEmpty());

System.out.println("\n=== PROBAR UNDERFLOW ===");

try {
    cola.dequeue();
```

```
        } catch (java.util.NoSuchElementException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
=== ENCOLAR ELEMENTOS ===
Frente -> 10 -> 20 -> 30 <- Final

=== CONSULTAR EL FRENTE ===
Frente: 10

=== BUSCAR ELEMENTOS ===
¿Contiene 20? true
¿Contiene 99? false

=== DESENCOLAR ELEMENTOS ===
Sale: 10
Frente -> 20 -> 30 <- Final
Sale: 20
Frente -> 30 <- Final
Sale: 30
Cola vacia.

¿La cola esta vacia? true

=== PROBAR UNDERFLOW ===
```

Underflow: no se puede desencolar una cola vacia.

Ejercicio 3

Necesito un algoritmo para Java para transformar la pila enlazada en una estructura generica PilaEnlazada<T> que pueda almacenar cualquier tipo de dato, como por ejemplo Integer, String u objetos propios creados por el usuario. Usar tipos genericos en Java permite crear clases reutilizables con seguridad de tipos en tiempo de compilacion sin tener que duplicar codigo para cada clase ni hacer casteos. La logica algoritmica de la pila no depende en absoluto del tipo de dato almacenado, ya que el encadenamiento de nodos y la politica LIFO operan unicamente manipulando referencias en la cabeza sin importar que informacion viaja adentro del nodo. La pila debe implementar push, pop, peek, isEmpty, contains e imprimir. Casos especiales: control de pila vacia en pop/peek, y en el metodo contains usar obligatoriamente .equals() para comparar objetos en vez de ==. Probar con una pila de Integer, una de String y una con una clase propia como Persona.

```
import java.util.Objects;

import java.util.NoSuchElementException;


// ====================================

// Pila enlazada generica

// ====================================

class PilaEnlazada<T> {


    // Nodo generico: puede almacenar cualquier tipo T

    private static class Nodo<T> {

        T dato;

        Nodo<T> sig;


        public Nodo(T dato) {

            this.dato = dato;

            this.sig = null;
```

```
    }
}

// Apunta al tope de la pila
private Nodo<T> head;

public PilaEnlazada() {
    head = null;
}

// Agrega un elemento al tope: O(1)
public void push(T elemento) {
    Nodo<T> nuevo = new Nodo<>(elemento);

    // El nuevo nodo apunta al antiguo tope
    nuevo.sig = head;

    // El nuevo nodo pasa a ser el tope
    head = nuevo;
}

// Quita y devuelve el tope: O(1)
public T pop() {
    if (isEmpty()) {
        throw new NoSuchElementException(
            "Underflow: la pila esta vacia."
```

```
            );
        }

        T elemento = head.dato;

        // El siguiente nodo pasa a ser el tope
        head = head.sig;

        return elemento;
    }

    // Consulta el tope sin quitarlo: O(1)
    public T peek() {
        if (isEmpty()) {
            throw new NoSuchElementException(
                "Underflow: no se puede consultar una pila vacia."
            );
        }

        return head.dato;
    }

    // Comprueba si la pila esta vacia: O(1)
    public boolean isEmpty() {
        return head == null;
    }
```

```
// Busca un elemento: O(n)
public boolean contains(T elemento) {
    Nodo<T> actual = head;

    while (actual != null) {

        // equals compara el contenido de los objetos
        if (Objects.equals(actual.dato, elemento)) {
            return true;
        }

        actual = actual.sig;
    }

    return false;
}

// Imprime desde el tope hacia la base: O(n)
public void imprimir() {
    if (isEmpty()) {
        System.out.println("Pila vacia.");
        return;
    }

    Nodo<T> actual = head;
```

```
        System.out.println("Tope");

        while (actual != null) {
            System.out.println("| " + actual.dato + " |");
            actual = actual.sig;
        }

        System.out.println("-----");
        System.out.println("Base");
    }
}

// ====================================
// Clase propia Persona
// ====================================
class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
```

```
    return nombre;
}

public int getEdad() {
    return edad;
}

@Override
public String toString() {
    return nombre + " (" + edad + " años)";
}

// Define cuando dos personas se consideran iguales
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }

    if (!(obj instanceof Persona otra)) {
        return false;
    }

    return edad == otra.edad
        && Objects.equals(nombre, otra.nombre);
}
```

```
    @Override
    public int hashCode() {
        return Objects.hash(nombre, edad);
    }
}

// ====================================
// Pruebas
// ====================================
public class Main {

    public static void main(String[] args) {

        // --------------------------------
        // 1. Pila de Integer
        // --------------------------------
        System.out.println("=== PILA DE INTEGER ===");

        PilaEnlazada<Integer> numeros = new PilaEnlazada<>();

        numeros.push(10);
        numeros.push(20);
        numeros.push(30);

        numeros.imprimir();
```

```
System.out.println("Tope: " + numeros.peek());
System.out.println("Contiene 20: " + numeros.contains(20));
System.out.println("Contiene 99: " + numeros.contains(99));

System.out.println("Sale: " + numeros.pop());

numeros.imprimir();

// ---------------------------------
// 2. Pila de String
// ---------------------------------
System.out.println("\n=== PILA DE STRING ===");

PilaEnlazada<String> palabras = new PilaEnlazada<>();

palabras.push("Java");
palabras.push("Python");
palabras.push("C");

palabras.imprimir();

System.out.println("Tope: " + palabras.peek());
System.out.println(
    "Contiene Java: " + palabras.contains("Java")
);
```

```
// ---------------------------------
// 3. Pila de Persona
// ---------------------------------
System.out.println("\n=== PILA DE PERSONA ===");

PilaEnlazada<Persona> personas = new PilaEnlazada<>();

Persona ana = new Persona("Ana", 20);
Persona luis = new Persona("Luis", 25);
Persona otraAna = new Persona("Ana", 20);

personas.push(ana);
personas.push(luis);

personas.imprimir();

System.out.println("Tope: " + personas.peek());

// otraAna es otro objeto, pero tiene los mismos datos que ana
System.out.println(
    "Contiene otra Ana: " + personas.contains(otraAna)
);

System.out.println("Sale: " + personas.pop());
```

```
        personas.imprimir();

        // --------------------------------
        // 4. Probar pila vacia
        // --------------------------------
        System.out.println("\n=== CONTROL DE PILA VACIA ===");

        PilaEnlazada<Integer> vacia = new PilaEnlazada<>();

        try {
            vacia.pop();
        } catch (NoSuchElementException e) {
            System.out.println(e.getMessage());
        }

        try {
            vacia.peek();
        } catch (NoSuchElementException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Ejercicio 4

Se requiere implementar en Java una clase ColaEnlazada<T> utilizando tipos genericos para que pueda encolar y desencolar cualquier tipo de objeto. Se mantiene inalterable el comportamiento FIFO independientemente de que los nodos almacenen textos simples o clases de negocio complejas, ya que la cola solo administra el orden cronologico de llegada

a traves de los enlaces head y tail. Debe implementar enqueue, dequeue, front, isEmpty, contains e imprimir. Casos especiales: vaciamiento de la cola donde tail debe quedar en null al salir el ultimo nodo, y chequeo de excepciones al pedir front o dequeue en cola vacia. La comparacion en contains debe hacerse con .equals(). El programa debe probarse instanciando una cola de nombres de personas (String) y otra cola con objetos de una clase Cliente (con atributos como id y nombre).

Ejercicio 5

Quiero implementar en Java un simulador basico de historial de navegacion web utilizando una pila enlazada. Corresponde usar una estructura de pila y no una cola porque el boton "Atrás" de cualquier navegador funciona estrictamente bajo la regla LIFO, ya que la ultima pagina web que visito el usuario es exactamente la primera a la que necesita regresar cuando decide volver hacia atras. El sistema debe permitir: visitarPagina(String url), volverAtras(), paginaActual() e imprimirHistorial(). Internamente debe apoyarse en nodos enlazados donde la pagina activa es la cabeza de la lista. Casos especiales: si el usuario intenta volver atras cuando ya no quedan paginas previas en el historial, debe mostrar un mensaje amigable o lanzar excepcion indicando que esta en la pagina inicial sin romper el programa. Probar visitando 3 paginas, consultando la actual, retrocediendo dos veces y mostrando el historial resultante.

```
import java.util.NoSuchElementException;

import java.util.Objects;


// Nodo genérico

class Nodo<T> {

    T dato;

    Nodo<T> sig;


    public Nodo(T dato) {

        this.dato = dato;

        this.sig = null;

    }

}
```

```
// Cola enlazada genérica
class ColaEnlazada<T> {
    private Nodo<T> head;
    private Nodo<T> tail;

    public ColaEnlazada() {
        head = null;
        tail = null;
    }

    // Agrega un elemento al final de la cola
    public void enqueue(T dato) {
        Nodo<T> nuevo = new Nodo<>(dato);

        // Si está vacía, el nuevo nodo es el primero y el último
        if (isEmpty()) {
            head = nuevo;
            tail = nuevo;
        } else {
            // El último nodo apunta al nuevo
            tail.sig = nuevo;
            tail = nuevo;
        }
    }

    // Elimina y devuelve el primer elemento
```

```
public T dequeue() {
    if (isEmpty()) {
        throw new NoSuchElementException(
            "No se puede desencolar: la cola está vacía."
        );
    }

    T dato = head.dato;
    head = head.sig;

    // Si eliminamos el último nodo, tail también debe quedar en null
    if (head == null) {
        tail = null;
    }

    return dato;
}

// Devuelve el primer elemento sin eliminarlo
public T front() {
    if (isEmpty()) {
        throw new NoSuchElementException(
            "No se puede consultar el frente: la cola está vacía."
        );
    }
```

```
    return head.dato;
}

// Indica si la cola está vacía
public boolean isEmpty() {
    return head == null;
}

// Busca un elemento usando equals()
public boolean contains(T dato) {
    Nodo<T> actual = head;

    while (actual != null) {
        if (Objects.equals(actual.dato, dato)) {
            return true;
        }

        actual = actual.sig;
    }

    return false;
}

// Imprime los elementos desde el primero hasta el último
public void imprimir() {
    Nodo<T> actual = head;
```

```
        System.out.print("Frente -> ");

        while (actual != null) {
            System.out.print(actual.dato);

            if (actual.sig != null) {
                System.out.print(" -> ");
            }

            actual = actual.sig;
        }

        System.out.println(" -> null");
    }
}

// Clase de negocio
class Cliente {
    private int id;
    private String nombre;

    public Cliente(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }
```

```
public int getId() {
    return id;
}

public String getNombre() {
    return nombre;
}

// Dos clientes son iguales si tienen el mismo id
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }

    if (!(obj instanceof Cliente)) {
        return false;
    }

    Cliente otro = (Cliente) obj;
    return this.id == otro.id;
}

@Override
public int hashCode() {
```

```
        return Objects.hash(id);
    }

    @Override
    public String toString() {
        return "Cliente{id=" + id + ", nombre='" + nombre + "'}";
    }
}

public class Main {
    public static void main(String[] args) {

        // PRUEBA 1: Cola de nombres (String)
        System.out.println("=== COLA DE NOMBRES ===");

        ColaEnlazada<String> nombres = new ColaEnlazada<>();

        nombres.enqueue("Ana");
        nombres.enqueue("Bruno");
        nombres.enqueue("Carla");

        nombres.imprimir();

        System.out.println("Primero: " + nombres.front());
        System.out.println("¿Está Bruno? " + nombres.contains("Bruno"));
```

```
System.out.println("Sale: " + nombres.dequeue());
nombres.imprimir();

// PRUEBA 2: Cola de clientes
System.out.println("\n=== COLA DE CLIENTES ===");

ColaEnlazada<Cliente> clientes = new ColaEnlazada<>();

Cliente c1 = new Cliente(1, "Lucía");
Cliente c2 = new Cliente(2, "Mateo");
Cliente c3 = new Cliente(3, "Sofía");

clientes.enqueue(c1);
clientes.enqueue(c2);
clientes.enqueue(c3);

clientes.imprimir();

System.out.println("Primero: " + clientes.front());

// Se crea otro objeto con el mismo id que c2
Cliente buscado = new Cliente(2, "Otro nombre");
System.out.println(
    "¿Está el cliente con id 2? " + clientes.contains(buscado)
);
```

```
        System.out.println("Sale: " + clientes.dequeue());
        clientes.imprimir();

        // PRUEBA 3: Vaciar la cola y comprobar el estado
        System.out.println("\n=== VACIAR LA COLA ===");

        while (!clientes.isEmpty()) {
            System.out.println("Sale: " + clientes.dequeue());
        }

        System.out.println("¿Está vacía? " + clientes.isEmpty());

        // PRUEBA 4: Excepciones al consultar una cola vacía
        try {
            clientes.front();
        } catch (NoSuchElementException e) {
            System.out.println("Excepción en front(): " + e.getMessage());
        }

        try {
            clientes.dequeue();
        } catch (NoSuchElementException e) {
            System.out.println("Excepción en dequeue(): " + e.getMessage());
        }
    }
}
```

=== COLA DE NOMBRES ===

Frente -> Ana -> Bruno -> Carla -> null

Primero: Ana

¿Está Bruno? true

Sale: Ana

Frente -> Bruno -> Carla -> null

=== COLA DE CLIENTES ===

Frente -> Cliente{id=1, nombre='Lucía'} -> Cliente{id=2, nombre='Mateo'} -> Cliente{id=3, nombre='Sofía'} -> null

Primero: Cliente{id=1, nombre='Lucía'}

¿Está el cliente con id 2? true

Sale: Cliente{id=1, nombre='Lucía'}

Frente -> Cliente{id=2, nombre='Mateo'} -> Cliente{id=3, nombre='Sofía'} -> null

=== VACIAR LA COLA ===

Sale: Cliente{id=2, nombre='Mateo'}

Sale: Cliente{id=3, nombre='Sofía'}

¿Está vacía? true

Excepción en front(): No se puede consultar el frente: la cola está vacía.

Excepción en dequeue(): No se puede desencolar: la cola está vacía.

Ejercicio 6

Necesito implementar en Java un sistema de atencion al cliente para una sucursal de un banco usando una cola enlazada. Cada cliente debe modelarse como un objeto con los atributos: nombre, numero de turno y motivo de consulta. La estructura correcta es una cola (Queue) y no una pila porque la atencion bancaria exige cumplir con la politica FIFO garantizando que el primer cliente que llego a la fila sea el primero en ser llamado a la caja.

El sistema debe implementar: agregarCliente(Cliente c), atenderCliente(), proximoCliente() e imprimirFila(). Casos especiales: intentar atender o consultar cuando la fila esta vacia, y la correcta puesta a null del puntero tail cuando se atiende al ultimo cliente en espera. Probar cargando 3 clientes con distintos motivos, atendiendo al primero y mostrando los que quedan en espera.

```
import java.util.NoSuchElementException;

// Representa a un cliente del banco
class Cliente {
    private String nombre;
    private int numeroTurno;
    private String motivoConsulta;

    public Cliente(String nombre, int numeroTurno, String motivoConsulta) {
        this.nombre = nombre;
        this.numeroTurno = numeroTurno;
        this.motivoConsulta = motivoConsulta;
    }

    public String getNombre() {
        return nombre;
    }

    public int getNumeroTurno() {
        return numeroTurno;
    }
```

```
    public String getMotivoConsulta() {
        return motivoConsulta;
    }

    @Override
    public String toString() {
        return "Turno " + numeroTurno
                + " - " + nombre
                + " - Motivo: " + motivoConsulta;
    }
}

// Representa cada nodo de la cola
class Nodo {
    Cliente dato;
    Nodo sig;

    public Nodo(Cliente dato) {
        this.dato = dato;
        this.sig = null;
    }
}

// Cola enlazada para la atención bancaria
class ColaBanco {
    private Nodo head;
```

```
    private Nodo tail;

    public ColaBanco() {
        head = null;
        tail = null;
    }

    // Agrega un cliente al final de la fila
    public void agregarCliente(Cliente c) {
        Nodo nuevo = new Nodo(c);

        if (head == null) {
            // Si la cola está vacía, es el primero y el último
            head = nuevo;
            tail = nuevo;
        } else {
            tail.sig = nuevo;
            tail = nuevo;
        }

        System.out.println("Cliente agregado: " + c);
    }

    // Atiende y elimina al primer cliente de la fila
    public Cliente atenderCliente() {
        if (head == null) {
```

```
            throw new NoSuchElementException(
                "No hay clientes para atender."
            );
        }

        Cliente atendido = head.dato;
        head = head.sig;

        // Si se atendió al último cliente, la cola queda vacía
        if (head == null) {
            tail = null;
        }

        return atendido;
    }

    // Consulta quién será atendido sin quitarlo de la fila
    public Cliente proximoCliente() {
        if (head == null) {
            throw new NoSuchElementException(
                "No hay clientes esperando."
            );
        }

        return head.dato;
    }
```

```
    // Muestra todos los clientes en orden FIFO
    public void imprimirFila() {
        if (head == null) {
            System.out.println("La fila está vacía.");
            return;
        }

        Nodo actual = head;

        System.out.println("Fila de espera:");

        while (actual != null) {
            System.out.println(actual.dato);
            actual = actual.sig;
        }
    }

    public boolean estaVacia() {
        return head == null;
    }
}

public class Main {
    public static void main(String[] args) {
        ColaBanco banco = new ColaBanco();
```

```
// Cargamos tres clientes con diferentes consultas
Cliente c1 = new Cliente(
    "Lucía", 1, "Depósito"
);

Cliente c2 = new Cliente(
    "Mateo", 2, "Retiro de dinero"
);

Cliente c3 = new Cliente(
    "Sofía", 3, "Consulta de saldo"
);

banco.agregarCliente(c1);
banco.agregarCliente(c2);
banco.agregarCliente(c3);

System.out.println("\n--- Estado inicial ---");
banco.imprimirFila();

System.out.println("\n--- Próximo cliente ---");
System.out.println(banco.proximoCliente());

System.out.println("\n--- Atención en caja ---");
Cliente atendido = banco.atenderCliente();
```

```
System.out.println("Atendiendo a: " + atendido);

System.out.println("\n--- Clientes que quedan en espera ---");
banco.imprimirFila();

// Prueba opcional: atender hasta vaciar la fila
System.out.println("\n--- Vaciando la fila ---");

while (!banco.estaVacia()) {
    System.out.println("Atendiendo a: "
            + banco.atenderCliente());
}

banco.imprimirFila();

// Comprobamos las excepciones en una fila vacía
System.out.println("\n--- Prueba de excepciones ---");

try {
    banco.proximoCliente();
} catch (NoSuchElementException e) {
    System.out.println("Error: " + e.getMessage());
}

try {
    banco.atenderCliente();
```

```
        } catch (NoSuchElementException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
--- Atención en caja ---
Atendiendo a: Turno 1 - Lucía - Motivo: Depósito

--- Clientes que quedan en espera ---
Fila de espera:
Turno 2 - Mateo - Motivo: Retiro de dinero
Turno 3 - Sofía - Motivo: Consulta de saldo
```

Ejercicio 7

Quiero implementar un programa en Java que verifique si una expresion matematica o cadena de texto tiene sus parentesis correctamente balanceados utilizando una pila enlazada. La pila es la estructura ideal porque los parentesis operan de forma anidada y el ultimo parentesis que se abre debe ser obligatoriamente el primero en cerrarse (comportamiento LIFO). El funcionamiento interno consiste en recorrer caracter por caracter: cada vez que encuentra una apertura '(' hace push a la pila, y cuando encuentra un cierre ')' hace pop para emparejarlo. Casos especiales a contemplar: si se intenta hacer pop y la pila esta vacia (cierre de mas), o si al terminar de procesar toda la cadena la pila todavia contiene elementos (apertura sin cerrar), la expresion es invalida; solo es valida si termina completamente vacia (isEmpty()). Probar con ejemplos validos como (2 + 3) * (5 - 1) o ((a + b) * c), e invalidos como (2 + 3 y ())(.

```
import java.util.NoSuchElementException;

// Nodo de la pila enlazada
```

```
class Nodo {
    char dato;
    Nodo sig;

    public Nodo(char dato) {
        this.dato = dato;
        this.sig = null;
    }
}

// Pila enlazada de caracteres
class PilaEnlazada {
    private Nodo head;

    public PilaEnlazada() {
        head = null;
    }

    // Agrega un carácter en la cima de la pila
    public void push(char dato) {
        Nodo nuevo = new Nodo(dato);

        nuevo.sig = head;
        head = nuevo;
    }
```

```
    // Elimina y devuelve el carácter de la cima
    public char pop() {
        if (isEmpty()) {
            throw new NoSuchElementException(
                "No se puede hacer pop: la pila está vacía."
            );
        }

        char dato = head.dato;
        head = head.sig;

        return dato;
    }

    // Indica si la pila está vacía
    public boolean isEmpty() {
        return head == null;
    }
}

// Verifica si los paréntesis están balanceados
class VerificadorParentesis {

    public static boolean estaBalanceada(String expresion) {
        PilaEnlazada pila = new PilaEnlazada();
```

```
        // Recorremos la expresión carácter por carácter
        for (int i = 0; i < expresion.length(); i++) {
            char caracter = expresion.charAt(i);

            if (caracter == '(') {
                // Si abre un paréntesis, lo guardamos
                pila.push(caracter);

            } else if (caracter == ')') {
                // Si cierra, debe existir una apertura pendiente
                if (pila.isEmpty()) {
                    return false;
                }

                // Emparejamos el cierre con la última apertura
                pila.pop();
            }
        }

        // Es válida únicamente si no quedaron aperturas pendientes
        return pila.isEmpty();
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        String[] expresiones = {
            "(2 + 3) * (5 - 1)",
            "((a + b) * c)",
            "(2 + 3",
            "())(",
            "()",
            "",
            "((x))"
        };

        for (String expresion : expresiones) {
            boolean resultado =
                VerificadorParentesis.estaBalanceada(expresion);

            System.out.println(
                "\"" + expresion + "\" -> "
                + (resultado ? "VÁLIDA" : "INVÁLIDA")
            );
        }
    }
}
```

"(2 + 3) * (5 - 1)" -> VÁLIDA

"((a + b) * c)" -> VÁLIDA

"(2 + 3" -> INVÁLIDA

"())(" -> INVÁLIDA

"()" -> VÁLIDA

"" -> VÁLIDA

"((x))" -> VÁLIDA

Ejercicio 8

Se requiere un programa en Java que invierta el orden de las letras de una palabra o texto utilizando una pila enlazada. La politica LIFO permite invertir una secuencia de datos de manera natural: al apilar los caracteres en el orden original de izquierda a derecha, el primer caracter leido queda en el fondo y el ultimo queda en el tope; al desapilar todos los elementos sucesivamente con pop, salen en sentido exactamente inverso. El algoritmo debe recorrer la palabra haciendo push de cada letra en una pila de caracteres y luego ir desapilando concatenando el resultado en un nuevo String. Casos especiales: cadenas vacias, palabras de un solo caracter o con espacios. Probar invirtiendo la palabra "algoritmo" para verificar que devuelva "omtirogla", y probar tambien con una palabra corta como "java".

```
import java.util.NoSuchElementException;


class Nodo {
    char dato;
    Nodo sig;


    public Nodo(char dato) {
        this.dato = dato;
        this.sig = null;
    }
}


class PilaEnlazada {
```

```
private Nodo head;

public PilaEnlazada() {
    head = null;
}

// Apila un carácter en el tope
public void push(char dato) {
    Nodo nuevo = new Nodo(dato);
    nuevo.sig = head;
    head = nuevo;
}

// Desapila y devuelve el carácter del tope
public char pop() {
    if (isEmpty()) {
        throw new NoSuchElementException(
            "La pila está vacía."
        );
    }

    char dato = head.dato;
    head = head.sig;

    return dato;
}
```

```
    public boolean isEmpty() {
        return head == null;
    }
}

public class Main {

    public static String invertir(String texto) {
        PilaEnlazada pila = new PilaEnlazada();

        // 1. Apilar cada carácter de izquierda a derecha
        for (int i = 0; i < texto.length(); i++) {
            pila.push(texto.charAt(i));
        }

        // 2. Desapilar y construir el texto invertido
        StringBuilder invertido = new StringBuilder();

        while (!pila.isEmpty()) {
            invertido.append(pila.pop());
        }

        return invertido.toString();
    }
```

```
    public static void main(String[] args) {
        System.out.println(invertir("algoritmo"));
        System.out.println(invertir("java"));
        System.out.println(invertir("a"));
        System.out.println(invertir(""));
        System.out.println(invertir("hola mundo"));
    }
}
```

omtirogla

avaj

a

odnum aloh

Ejercicio 9

Quiero implementar en Java un simulador de cola de impresion para una oficina utilizando una cola enlazada con head y tail. Cada trabajo a imprimir debe representarse con un objeto que contenga: nombre del archivo, cantidad de paginas y el usuario que lo envio. Una cola de impresion debe procesar los documentos usando FIFO (First In, First Out): para garantizar la equidad y respetar el orden cronologico de envio, evitando que un archivo enviado mas tarde se imprima antes que uno que ya estaba esperando en la cola del servidor. El sistema debe permitir: agregarTrabajo(Trabajo t), imprimirProximo(), consultarProximo() y mostrarPendientes(). Casos especiales: intentar imprimir o consultar con la cola vacia, y reasignar tail = null cuando se termina de imprimir el ultimo documento de la fila.Probar mandando 3 documentos de distintos usuarios, imprimiendo el primero y listando los restantes.

```
import java.util.NoSuchElementException;

// Representa un trabajo de impresión
```

```
class Trabajo {
    private String nombreArchivo;
    private int cantidadPaginas;
    private String usuario;

    public Trabajo(String nombreArchivo, int cantidadPaginas,
                String usuario) {
        this.nombreArchivo = nombreArchivo;
        this.cantidadPaginas = cantidadPaginas;
        this.usuario = usuario;
    }

    public String getNombreArchivo() {
        return nombreArchivo;
    }

    public int getCantidadPaginas() {
        return cantidadPaginas;
    }

    public String getUsuario() {
        return usuario;
    }

    @Override
    public String toString() {
```

```
        return "Archivo: " + nombreArchivo
                + " | Páginas: " + cantidadPaginas
                + " | Usuario: " + usuario;
    }
}

// Nodo que almacena un trabajo y referencia al siguiente
class Nodo {
    Trabajo dato;
    Nodo sig;

    public Nodo(Trabajo dato) {
        this.dato = dato;
        this.sig = null;
    }
}

// Cola enlazada de impresión
class ColaImpresion {
    private Nodo head;
    private Nodo tail;

    public ColaImpresion() {
        head = null;
        tail = null;
    }
```

```
// Agrega un trabajo al final de la cola
public void agregarTrabajo(Trabajo t) {
    Nodo nuevo = new Nodo(t);

    if (head == null) {
        // La cola está vacía: el nodo es el primero y el último
        head = nuevo;
        tail = nuevo;
    } else {
        // El último nodo enlaza al nuevo
        tail.sig = nuevo;
        tail = nuevo;
    }

    System.out.println("Trabajo agregado: " + t);
}

// Imprime y elimina el primer trabajo de la cola
public Trabajo imprimirProximo() {
    if (head == null) {
        throw new NoSuchElementException(
            "No hay trabajos pendientes para imprimir."
        );
    }
```

```
    Trabajo impreso = head.dato;
    head = head.sig;

    // Si no quedan nodos, ambos punteros deben quedar en null
    if (head == null) {
        tail = null;
    }

    return impreso;
}

// Consulta el siguiente trabajo sin eliminarlo
public Trabajo consultarProximo() {
    if (head == null) {
        throw new NoSuchElementException(
            "No hay trabajos pendientes en la cola."
        );
    }

    return head.dato;
}

// Muestra todos los trabajos en el orden de llegada
public void mostrarPendientes() {
    if (head == null) {
        System.out.println("No hay trabajos pendientes.");
```

```
            return;
        }

        Nodo actual = head;

        System.out.println("Trabajos pendientes:");

        while (actual != null) {
            System.out.println(actual.dato);
            actual = actual.sig;
        }
    }

    public boolean estaVacia() {
        return head == null;
    }
}

public class Main {
    public static void main(String[] args) {
        ColaImpresion cola = new ColaImpresion();

        // Enviamos tres documentos de distintos usuarios
        Trabajo t1 = new Trabajo(
            "Informe.pdf", 10, "Lucía"
        );
```

```
Trabajo t2 = new Trabajo(
    "TrabajoPractico.docx", 5, "Mateo"
);

Trabajo t3 = new Trabajo(
    "Presentacion.pptx", 20, "Sofía"
);

cola.agregarTrabajo(t1);
cola.agregarTrabajo(t2);
cola.agregarTrabajo(t3);

System.out.println("\n--- Próximo trabajo ---");
System.out.println(cola.consultarProximo());

System.out.println("\n--- Imprimiendo documento ---");
Trabajo impreso = cola.imprimirProximo();
System.out.println("Impreso: " + impreso);

System.out.println("\n--- Documentos restantes ---");
cola.mostrarPendientes();

// Prueba de excepciones con la cola vacía
System.out.println("\n--- Prueba de cola vacía ---");
```

```
        ColaImpresion vacia = new ColaImpresion();

        try {
            vacia.consultarProximo();
        } catch (NoSuchElementException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            vacia.imprimirProximo();
        } catch (NoSuchElementException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
Trabajo agregado: Archivo: Informe.pdf | Páginas: 10 | Usuario: Lucía
Trabajo agregado: Archivo: TrabajoPractico.docx | Páginas: 5 | Usuario: Mateo
Trabajo agregado: Archivo: Presentacion.pptx | Páginas: 20 | Usuario: Sofía
--- Imprimiendo documento ---
Impreso: Archivo: Informe.pdf | Páginas: 10 | Usuario: Lucía

--- Documentos restantes ---
Trabajos pendientes:
Archivo: TrabajoPractico.docx | Páginas: 5 | Usuario: Mateo
Archivo: Presentacion.pptx | Páginas: 20 | Usuario: Sofía
```

Ejercicio 10

Necesito implementar en Java una lista doblemente enlazada generica ListaDoble<T> sin usar estructuras nativas de Java. Cada nodo doble tiene tres atributos: la carga util dato, y dos referencias de enlace que son siguiente (apunta al nodo posterior) y anterior (apunta al nodo previo). Se necesitan ambas referencias porque permiten recorrer la coleccion en ambos sentidos de forma bidireccional sin tener que reiniciar desde el inicio como en la lista simple. La lista debe implementar: insertarAlInicio(T dato), insertarAlFinal(T dato), eliminar(T dato), imprimirHaciaAdelante() e imprimirHaciaAtras(). Casos especiales a contemplar: al eliminar un nodo, si es el unico nodo de la lista, si es la cabeza (actualizar head y anular el anterior del nuevo primero), si es el ultimo nodo (anular el siguiente del nuevo ultimo) o si esta en el medio reconectando los punteros de los nodos vecinos para no romper la doble cadena. Probar insertando elementos, imprimiendo hacia adelante y hacia atras para verificar ambos enlaces, y borrando nodos de las tres posiciones posibles.

```
import java.util.Objects;

// Nodo doblemente enlazado genérico
class NodoDoble<T> {
    T dato;
    NodoDoble<T> siguiente;
    NodoDoble<T> anterior;

    public NodoDoble(T dato) {
        this.dato = dato;
        this.siguiente = null;
        this.anterior = null;
    }
}

// Lista doblemente enlazada genérica
class ListaDoble<T> {
    private NodoDoble<T> head;

    public ListaDoble() {
        head = null;
    }

    // Inserta un elemento al principio
    public void insertarAlInicio(T dato) {
        NodoDoble<T> nuevo = new NodoDoble<>(dato);

        if (head == null) {
            // Si está vacía, el nuevo nodo es el único
            head = nuevo;
```

```
    } else {
        // El nuevo nodo apunta al que era el primero
        nuevo.siguiente = head;

        // El antiguo primero apunta hacia el nuevo
        head.anterior = nuevo;

        // Actualizamos la cabeza
        head = nuevo;
    }
}

// Inserta un elemento al final
public void insertarAlFinal(T dato) {
    NodoDoble<T> nuevo = new NodoDoble<>(dato);

    if (head == null) {
        // Si está vacía, el nuevo nodo es el primero
        head = nuevo;
    } else {
        // Buscamos el último nodo
        NodoDoble<T> actual = head;

        while (actual.siguiente != null) {
            actual = actual.siguiente;
        }

        // Conectamos el último nodo con el nuevo
        actual.siguiente = nuevo;
        nuevo.anterior = actual;
    }
}

// Elimina la primera aparición de un dato.
// Devuelve true si lo encontró y false si no existe.
public boolean eliminar(T dato) {
    NodoDoble<T> actual = head;

    // Buscamos el nodo que contiene el dato
    while (actual != null
            && !Objects.equals(actual.dato, dato)) {
        actual = actual.siguiente;
    }

    // Si llegamos al final, el dato no existe
    if (actual == null) {
        return false;
    }
```

```
    // CASO 1: el nodo es la cabeza
    if (actual == head) {
        head = actual.siguiente;

        // Si hay un nuevo primero, no debe tener anterior
        if (head != null) {
            head.anterior = null;
        }
    } else {
        // El nodo anterior salta al nodo que sigue al actual
        actual.anterior.siguiente = actual.siguiente;

        // CASO 2: el nodo está en el medio
        // o es el último nodo
        if (actual.siguiente != null) {
            actual.siguiente.anterior = actual.anterior;
        }
    }

    // Desconectamos el nodo eliminado
    actual.siguiente = null;
    actual.anterior = null;

    return true;
}

// Imprime desde el primero hasta el último
public void imprimirHaciaAdelante() {
    NodoDoble<T> actual = head;

    System.out.print("Inicio -> ");

    while (actual != null) {
        System.out.print(actual.dato + " -> ");
        actual = actual.siguiente;
    }

    System.out.println("null");
}

// Imprime desde el último hasta el primero
public void imprimirHaciaAtras() {
    NodoDoble<T> actual = head;

    // Primero llegamos al último nodo
    while (actual != null && actual.siguiente != null) {
        actual = actual.siguiente;
```

```
        }

        System.out.print("Final -> ");

        // Volvemos al inicio usando anterior
        while (actual != null) {
            System.out.print(actual.dato + " -> ");
            actual = actual.anterior;
        }

        System.out.println("null");
    }
}

public class Main {
    public static void main(String[] args) {
        ListaDoble<String> lista = new ListaDoble<>();

        // Insertamos elementos al final
        lista.insertarAlFinal("Ana");
        lista.insertarAlFinal("Bruno");
        lista.insertarAlFinal("Carla");
        lista.insertarAlFinal("Diego");
        lista.insertarAlFinal("Elena");

        // También probamos insertar al inicio
        lista.insertarAlInicio("Zoe");

        System.out.println("=== LISTA ORIGINAL ===");
        lista.imprimirHaciaAdelante();
        lista.imprimirHaciaAtras();

        // CASO 1: eliminar un nodo del medio
        System.out.println("\n=== ELIMINAR DEL MEDIO ===");
        System.out.println("¿Se eliminó Carla? "
                + lista.eliminar("Carla"));

        lista.imprimirHaciaAdelante();
        lista.imprimirHaciaAtras();

        // CASO 2: eliminar la cabeza
        System.out.println("\n=== ELIMINAR EL PRIMERO ===");
        System.out.println("¿Se eliminó Zoe? "
                + lista.eliminar("Zoe"));

        lista.imprimirHaciaAdelante();
        lista.imprimirHaciaAtras();
```

```
        // CASO 3: eliminar el último nodo
        System.out.println("\n=== ELIMINAR EL ÚLTIMO ===");
        System.out.println("¿Se eliminó Elena? "
                + lista.eliminar("Elena"));

        lista.imprimirHaciaAdelante();
        lista.imprimirHaciaAtras();

        // CASO 4: eliminar el único nodo
        System.out.println("\n=== ELIMINAR EL ÚNICO NODO ===");
        ListaDoble<Integer> unica = new ListaDoble<>();

        unica.insertarAlFinal(100);
        unica.imprimirHaciaAdelante();

        System.out.println("¿Se eliminó 100? "
                + unica.eliminar(100));

        unica.imprimirHaciaAdelante();
        unica.imprimirHaciaAtras();

        // CASO 5: dato inexistente
        System.out.println("\n=== DATO INEXISTENTE ===");
        System.out.println("¿Se eliminó Pedro? "
                + lista.eliminar("Pedro"));
    }
}
```

```
=== LISTA ORIGINAL ===
Inicio -> Zoe -> Ana -> Bruno -> Carla -> Diego -> Elena -> null
Final -> Elena -> Diego -> Carla -> Bruno -> Ana -> Zoe -> null

=== ELIMINAR DEL MEDIO ===
¿Se eliminó Carla? true
Inicio -> Zoe -> Ana -> Bruno -> Diego -> Elena -> null
Final -> Elena -> Diego -> Bruno -> Ana -> Zoe -> null

=== ELIMINAR EL PRIMERO ===
¿Se eliminó Zoe? true
Inicio -> Ana -> Bruno -> Diego -> Elena -> null
Final -> Elena -> Diego -> Bruno -> Ana -> null

=== ELIMINAR EL ÚLTIMO ===
¿Se eliminó Elena? true
Inicio -> Ana -> Bruno -> Diego -> null
Final -> Diego -> Bruno -> Ana -> null

=== ELIMINAR EL ÚNICO NODO ===
```

Inicio -> 100 -> null
¿Se eliminó 100? true
Inicio -> null
Final -> null